

Exame com consulta. Duração: 1h30m

Primeiro Mini-Teste

Group 1. Análise Lexical e Semântica (6 pts)

A CFG¹ G1 apresentada abaixo representa um subconjunto de uma gramática de “MiniC” (uma gramática para uma linguagem de programação focada num subconjunto da linguagem de programação C).

Alguns dos tokens para G1:

ID = [a-zA-Z][0-9a-zA-Z]*

NUM = [0-9]+

1.a) [1pt] Apresente o DFA para os tokens necessários para a gramática G1.**1.b)** [1pt] Apresente a árvore sintática concreta (CST) para o código exemplo abaixo e considerando a CFG G1.**Gramática G1:**

1. Function $\rightarrow$ Type ID (ArgList) CompoundStmt
2. ArgList $\rightarrow$ Arg | ArgList , Arg
3. Arg $\rightarrow$ Type ID
4. Type $\rightarrow$ void | int | float
5. Declaration $\rightarrow$ Type ID ;
6. CompoundStmt $\rightarrow$ { StmtList }
7. StmtList $\rightarrow$ StmtList Stmt | epsilon
8. Stmt $\rightarrow$ Expr ; | CompoundStmt | Declaration | ;
9. Expr $\rightarrow$ ID = Mag
10. Mag $\rightarrow$ Mag + Term | Mag - Term | Term
11. Term $\rightarrow$ Term * Factor | Term / Factor | Factor
12. Factor $\rightarrow$ (Mag) | - Factor | + Factor | ID | NUM

1.c) [1pt] Apresente uma possível árvore sintática abstrata (AST²) para a árvore sintática concreta do problema anterior.**1.d)** [3pt] Precisamos de estender a gramática para incluir: (a) enunciados *return*; (b) ciclos FOR e WHILE; e (c) construções IF. Defina os tokens e as modificações/extensões da gramática que sugere, indicando explicitamente se essas alterações são para (a), (b) ou (c). Para as comparações considere os comparadores seguintes: $=$ | $<$ | $>$ | $<=$ | $>=$ | $!=$.**Código exemplo:**

```
void f1(int a, float b) {  
    int c;  
    c = a*b;  
}
```

Group 2. Parsers LL (8 pts)**2.a)** [1pt] Indique os conjuntos *First* e *Follow* para as variáveis de gramática: ArgList, Mag, Term e Factor;**2.b)** [2pts] Mostre as linhas da tabela para o parser LL(1) considerando apenas as linhas relacionadas com as variáveis Expr, Mag e Term, e as colunas para os tokens cujas células não são vazias na secção da tabela

considerada;

2.c) [2pts] Tendo em conta a secção apresentada anteriormente da tabela do parser LL(1), pode-se concluir que a gramática é LL(1)? Porquê?**2.d)** [1pt] Modifique a secção da gramática entre as linhas 9 e 12 de forma a que essa secção possa ser implementada por um parser LL(1).**2.e)** [2pt] Mostre as funções necessárias, para as regras da gramática nas linhas 1 e 9 (isto é, para as variáveis *Function* e *Expr*), assim como o pseudo-código necessário para as implementar, como um parser descendente recursivo LL(1). Assuma que existe um analisador lexical que produz a sequência de tokens, a existência de uma variável global token, e uma função next() que produz o próximo token na sequência (no início a variável token corresponde ao primeiro token da sequência).**Group 3. Análise Semântica (6 pts)****3.a)** [3pts] Uma equipa está a trabalhar num novo compilador de Java e neste momento encontra-se a ponderar a análise semântica a ser incluída no compilador. Um dos aspetos que estão a ter em conta está relacionado com a inicialização de variáveis escalares e com o facto de estas terem de ser inicializadas antes de serem utilizadas. Descreva de forma sucinta a análise que pode ser incluída e os possíveis problemas que esta equipa pode enfrentar. Dê exemplos que ache adequados para lhes explicar os possíveis problemas.¹ Context-Free Grammar (Gramáticas Livres de Contexto)² Abstract Syntax Tree (Árvore Sintática Abstrata)

- 3.b) [3pts] Considere expressões aritméticas em Java. Escreva em pseudo-código a secção da análise semântica para verificar as regras semânticas relacionadas com os tipos das variáveis em expressões aritméticas que consistem apenas em constantes e variáveis escalares. Considere a existência de uma tabela de símbolos para o escopo onde a expressão se insere, uma AST que representa a expressão, os tipos de variáveis *int* e *float*, a existência de conversões *i2f* (*int2float*) e *f2i* (*float2int*), e * e + com as versões *float* (+_{float}, *_{float}) e *int* (+_{int}, *_{int}) associadas.

A análise semântica tem que considerar:

Input: AST de uma dada expressão aritmética e tabela de símbolos para o escopo dessa AST

Output: erro semântico ou uma árvore baseada na AST dada como entrada, mas estendida de acordo com o resultado da análise semântica

Utilize uma AST para a expressão: $a = b * c + b * 60$; com os seguintes tipos para as variáveis: a: float, b: int, c: float, e mostre o resultado da análise semântica que propôs no pseudo-código anterior.

(Fim.)